

Rapport Technique

Phase 2 : Construction du Dataset Edge-Centric

(Line Graph / Dual Graph)

Projet NYC Traffic Prediction

Prédiction de Trafic par Graph Neural Network

Date	5 janvier 2026
Version	1.0.0
Script	<code>build_line_graph.py</code>
Contrainte mémoire	8 GB RAM

Suite de la Phase 1 : Enrichissement et Normalisation

Contents

1	Introduction	2
1.1	Contexte	2
1.2	Problématique	2
1.3	Contrainte Technique : 8 GB RAM	2
2	Fondements Théoriques	2
2.1	Définition du Line Graph	2
2.2	Illustration	3
2.3	Propriétés Mathématiques	3
2.4	Avantages pour le GNN	3
3	Architecture du Script	4
3.1	Pipeline de Traitement	4
3.2	Gestion Mémoire : Stratégie en Deux Passes	4
3.3	Optimisation des Types de Données	5
4	Features des Nœuds du Line Graph	5
4.1	Encodage pour le GNN	6
5	Extraction des Labels	6
5.1	Problématique	6
5.2	Stratégie d'Agrégation	6
5.3	Fichiers de Sortie	7
6	Fichiers de Sortie	7
6.1	Structure du Répertoire	7
6.2	Format edge_index	7
6.3	Fichier de Métadonnées	7
7	Utilisation	8
7.1	Installation	8
7.2	Exécution	8
7.3	Arguments	8
8	Validation et Métriques	9
8.1	Vérifications Automatiques	9
8.2	Métriques de Qualité	9
9	Prochaines Étapes	9

9.1	Phase 3 : Architecture du Modèle	9
9.2	Phase 4 : Validation Inductive	9
10	Conclusion	10
A	Formules Mathématiques	10
A.1	Nombre d'Arêtes du Line Graph	10
A.2	Complexité Algorithmique	10
B	Code Source	11

1 Introduction

1.1 Contexte

Ce rapport documente l'implémentation de la **Phase 2** du projet de prédiction de trafic urbain : la transformation du graphe routier enrichi en **Line Graph** (également appelé Dual Graph ou Edge Graph).

La Phase 1 a produit des fichiers enrichis avec des features normalisées :

- `links_enriched.csv` : Segments routiers avec features topologiques
- `nodes_enriched.csv` : Intersections avec métriques de centralité
- `travel_times_enriched.csv` : Données temporelles (~22 GB)

1.2 Problématique

Les architectures GNN standard (GraphSAGE, GAT, GCN) prédisent naturellement sur les **nœuds** d'un graphe. Or, notre objectif est de prédire la congestion sur les **arêtes** (segments routiers).

Deux approches sont possibles :

1. **Approche directe** : Adapter l'architecture GNN pour prédire sur les arêtes (complexe)
2. **Approche Line Graph** : Transformer le graphe pour que les arêtes deviennent des nœuds (simple, élégant)

Nous adoptons l'approche Line Graph pour sa simplicité et sa compatibilité avec les frameworks existants (PyTorch Geometric).

1.3 Contrainte Technique : 8 GB RAM

Le fichier `travel_times_enriched.csv` fait environ 22 GB, ce qui dépasse largement la mémoire disponible. Le script est donc conçu avec une architecture **streaming** :

- Traitement par chunks de 200 000 lignes
- Sauvegarde incrémentale sur disque
- Libération explicite de mémoire (`gc.collect()`)
- Types de données optimisés (`float32`, `int32`)

2 Fondements Théoriques

2.1 Définition du Line Graph

Soit $G = (V, E)$ un graphe orienté où :

- V = ensemble des nœuds (intersections)

- E = ensemble des arêtes (segments routiers)

Le **Line Graph** $L(G) = (V', E')$ est défini par :

$$V' = E \quad (\text{les arêtes de } G \text{ deviennent les nœuds de } L(G)) \quad (1)$$

$$E' = \{(e_i, e_j) : \text{end}(e_i) = \text{begin}(e_j)\} \quad (2)$$

Autrement dit, deux nœuds du line graph sont connectés si les arêtes correspondantes dans le graphe original sont **consécutives** (l'une finit où l'autre commence).

2.2 Illustration

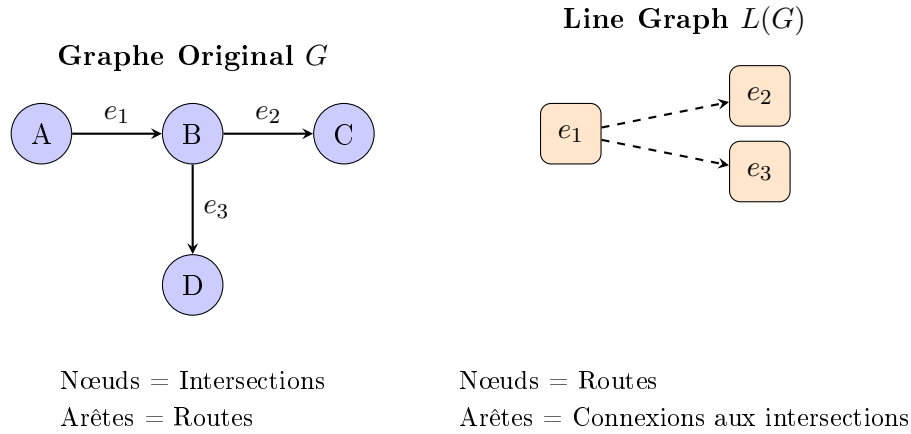


Figure 1: Transformation du graphe routier en Line Graph

2.3 Propriétés Mathématiques

Table 1: Correspondance entre graphe original et line graph

Propriété	Graphe Original G	Line Graph $L(G)$
Nœuds	$ V $ intersections	$ E $ segments
Arêtes	$ E $ segments	$\sum_{v \in V} d_{in}(v) \cdot d_{out}(v)$
Features nœuds	Centralité, degré	Hierarchie, longueur, capacité
Target	–	Temps de trajet (congestion)

2.4 Avantages pour le GNN

1. **Prédiction directe** : La target (travel_time) est naturellement sur les nœuds
2. **Message Passing** : Les routes voisines échangent des informations (effet de propagation du trafic)
3. **Compatibilité** : Format standard PyTorch Geometric
4. **Scalabilité** : Sparse matrix pour l'adjacence

3 Architecture du Script

3.1 Pipeline de Traitement

Le script `build_line_graph.py` suit un pipeline en 6 étapes conçu pour minimiser l'utilisation mémoire :

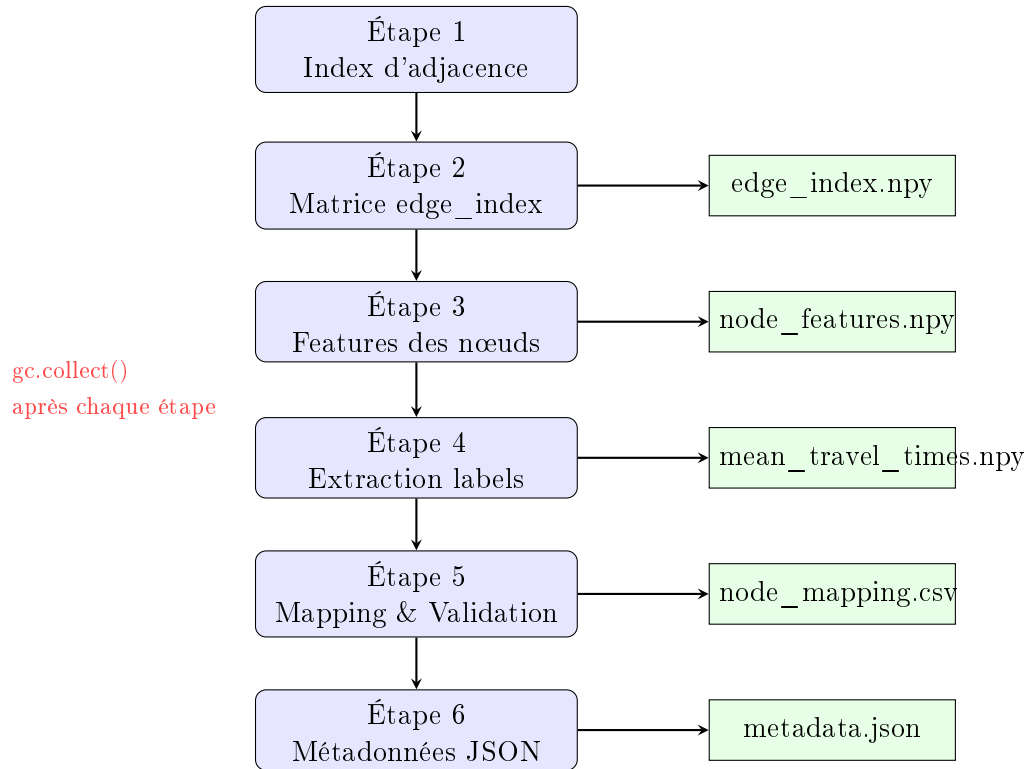


Figure 2: Pipeline de construction du Line Graph

3.2 Gestion Mémoire : Stratégie en Deux Passes

Pour construire l'adjacence sans charger tout en mémoire, nous utilisons deux passes sur le fichier :

Algorithm 1 Construction de l'adjacence en streaming

```

1: procedure BUILDADJACENCY(links_path)
2:   // Passe 1 : Construire les index
3:   node_to_outgoing  $\leftarrow \{\}$  ▷ node_id  $\rightarrow$  [link_ids sortants]
4:   node_to_incoming  $\leftarrow \{\}$  ▷ node_id  $\rightarrow$  [link_ids entrants]
5:   link_to_idx  $\leftarrow \{\}$  ▷ link_id  $\rightarrow$  index
6:   for all chunk in read_csv(links_path, chunksize=200k) do
7:     for all row in chunk do
8:       link_to_idx[row.link_id]  $\leftarrow$  idx
9:       node_to_outgoing[row.begin_node].append(row.link_id)
10:      node_to_incoming[row.end_node].append(row.link_id)
11:     end for
12:   end for
13:
14:   // Passe 2 : Générer les arêtes
15:   sources, targets  $\leftarrow [], []$ 
16:   for all chunk in read_csv(links_path, chunksize=200k) do
17:     for all row in chunk do
18:       end_node  $\leftarrow$  row.end_node_id
19:       for all target_link in node_to_outgoing[end_node] do
20:         if target_link  $\neq$  row.link_id then
21:           sources.append(link_to_idx[row.link_id])
22:           targets.append(link_to_idx[target_link])
23:         end if
24:       end for
25:     end for
26:   end for
27:   return np.array([sources, targets])
28: end procedure

```

3.3 Optimisation des Types de Données

Table 2: Types de données optimisés pour réduire l'empreinte mémoire

Donnée	Type	Taille	Justification
link_id	int64	8 bytes	Identifiant unique
edge_index	int32	4 bytes	Indices < 2 milliards
Features	float32	4 bytes	Précision suffisante
road_hierarchy	int8	1 byte	Valeurs 1-6
Binaires	int8	1 byte	0 ou 1

4 Features des Nœuds du Line Graph

Chaque nœud du line graph (= segment routier) possède un vecteur de features de dimension $d = 14$:

Table 3: Vecteur de features par nœud du line graph

Index	Feature	Type	Description
0	road_hierarchy	{1..6}	Classification hiérarchique universelle
1	length_percentile	[0, 1]	Rang percentile de la longueur
2	length_zscore	$\mathbb{R}$	Longueur normalisée (Z-score)
3	estimated_capacity	[0, 1]	Capacité relative estimée
4	begin_angle_sin	[-1, 1]	Orientation (composante sin)
5	begin_angle_cos	[-1, 1]	Orientation (composante cos)
6	angle_change	[0, 180]	Courbure du segment
7	start_centrality	[0, 1]	Centralité du nœud de départ
8	end_centrality	[0, 1]	Centralité du nœud d'arrivée
9	avg_centrality	[0, 1]	Centralité moyenne
10	centrality_gradient	[-1, 1]	Direction centre/périphérie
11	connects_center	{0, 1}	Segment proche du centre
12	start_degree_percentile	[0, 1]	Degré du nœud de départ
13	end_degree_percentile	[0, 1]	Degré du nœud d'arrivée

4.1 Encodage pour le GNN

Le vecteur de features $\mathbf{x}_i \in \mathbb{R}^{14}$ pour le nœud i du line graph est directement utilisable comme entrée du GNN :

$$\mathbf{X} = \begin{pmatrix} \mathbf{x}_1 \\ \mathbf{x}_2 \\ \vdots \\ \mathbf{x}_n \end{pmatrix} \in \mathbb{R}^{n \times 14} \quad (3)$$

où n est le nombre de segments routiers (nœuds du line graph).

5 Extraction des Labels

5.1 Problématique

Le fichier `travel_times_enriched.csv` contient ~ 22 GB de données avec des mesures horaires de temps de trajet. Pour l'entraînement initial, nous calculons une **moyenne globale** par lien.

5.2 Stratégie d'Agrégation

Pour chaque lien i , nous calculons :

$$\bar{t}_i = \frac{1}{n_i} \sum_{j=1}^{n_i} t_{i,j} \quad (4)$$

où $t_{i,j}$ est le j -ème temps de trajet observé pour le lien i et n_i le nombre d'observations.

5.3 Fichiers de Sortie

- `mean_travel_times.npy` : Vecteur $\bar{\mathbf{t}} \in \mathbb{R}^n$ des moyennes
- `observation_counts.npy` : Vecteur $\mathbf{n} \in \mathbb{N}^n$ des comptages

Le vecteur de comptages permet de pondérer la loss lors de l'entraînement :

$$\mathcal{L} = \frac{1}{\sum_i w_i} \sum_{i:n_i>0} w_i \cdot (\hat{t}_i - \bar{t}_i)^2 \quad (5)$$

avec $w_i = \min(n_i, n_{max})$ pour éviter la sur-pondération des liens très fréquentés.

6 Fichiers de Sortie

6.1 Structure du Répertoire

```

1 line_graph_data/
2 |-- edge_index.npy           # Matrice d'adjacence (2, num_edges)
3 |-- node_features.npy        # Features des noeuds (num_nodes, 14)
4 |-- node_mapping.csv         # Correspondance link_id <-> node_id
5 |-- mean_travel_times.npy     # Labels moyens (num_nodes,)
6 |-- observation_counts.npy    # Comptages (num_nodes,)
7 |-- line_graph_metadata.json

```

Listing 1: Structure de sortie

6.2 Format edge_index

Le format COO (Coordinate format) est le standard PyTorch Geometric :

```

1 import torch
2 import numpy as np
3 from torch_geometric.data import Data
4
5 # Charger les donnees
6 edge_index = torch.from_numpy(np.load('edge_index.npy')).long()
7 x = torch.from_numpy(np.load('node_features.npy')).float()
8 y = torch.from_numpy(np.load('mean_travel_times.npy')).float()
9
10 # Creer l'objet Data
11 data = Data(x=x, edge_index=edge_index, y=y)
12
13 print(f"Noeuds: {data.num_nodes}")
14 print(f"Aretes: {data.num_edges}")
15 print(f"Features: {data.num_node_features}")

```

Listing 2: Chargement avec PyTorch Geometric

6.3 Fichier de Métadonnées

```

1 {
2   "created_at": "2026-01-05T...",
3   "line_graph": {
4     "num_nodes": 350000,
5     "num_edges": 850000,
6     "num_features": 14,
7     "feature_names": ["road_hierarchy", "length_percentile", ...]
8   },
9   "validation": {
10    "is_valid": true,
11    "degree_stats": {
12      "out_degree_mean": 2.43,
13      "isolated_nodes": 1250
14    }
15  },
16  "labels": {
17    "coverage": 87.5,
18    "mean_observations_per_link": 156.3
19  }
20 }

```

Listing 3: Structure de line_graph_metadata.json

7 Utilisation

7.1 Installation

```

1 pip install numpy pandas scipy

```

7.2 Exécution

```

1 # Usage standard
2 python build_line_graph.py \
3   --data-dir ./enriched_data \
4   --output-dir ./line_graph_data
5
6 # Sans extraction des labels (plus rapide)
7 python build_line_graph.py \
8   -d ./enriched_data \
9   -o ./line_graph_data \
10  --skip-labels

```

7.3 Arguments

Table 4: Arguments du script

Argument	Défaut	Description
-d, -data-dir	./enriched_data	Répertoire des données Phase 1
-o, -output-dir	./line_graph_data	Répertoire de sortie
-skip-labels	False	Ne pas extraire les labels

8 Validation et Métriques

8.1 Vérifications Automatiques

Le script effectue plusieurs validations :

1. **Cohérence des dimensions** : `node_features.shape[0] == num_nodes`
2. **Validité des indices** : $\forall (i, j) \in E', i < n \wedge j < n$
3. **Absence de self-loops** : $\forall (i, j) \in E', i \neq j$
4. **Taux de NaN** dans les features

8.2 Métriques de Qualité

Table 5: Métriques attendues pour NYC

Métrique	Valeur attendue	Signification
Nombre de nœuds	~300k-400k	Segments routiers
Nombre d'arêtes	~700k-1M	Connexions aux intersections
Degré moyen	~2-3	Routes connectées par intersection
Nœuds isolés	< 5%	Segments sans connexion
Couverture labels	> 80%	Liens avec données de trafic

9 Prochaines Étapes

9.1 Phase 3 : Architecture du Modèle

Le line graph est maintenant prêt pour l'entraînement d'un GNN :

1. **Input Layer** : Projection des 14 features vers dimension cachée h

$$\mathbf{H}^{(0)} = \text{Linear}(\mathbf{X}) \in \mathbb{R}^{n \times h} \quad (6)$$

2. **Message Passing** ($\times 3$) : Agrégation des voisins

$$\mathbf{h}_i^{(l+1)} = \text{Update} \left(\mathbf{h}_i^{(l)}, \bigoplus_{j \in \mathcal{N}(i)} \text{Message}(\mathbf{h}_j^{(l)}) \right) \quad (7)$$

3. **Readout** : MLP pour la régression

$$\hat{t}_i = \text{MLP}(\mathbf{h}_i^{(L)}) \quad (8)$$

9.2 Phase 4 : Validation Inductive

Split spatial pour simuler le transfer learning :

- **Train** : Brooklyn, Manhattan (zones denses)

- **Validation** : Bronx (zone mixte)
- **Test** : Queens (zone périphérique, non vue)

Si la validation sur Queens est concluante, le transfert vers Marseille sera techniquement viable.

10 Conclusion

Ce rapport a présenté l'implémentation de la Phase 2 du projet de prédiction de trafic :

1. **Transformation Line Graph** : Les segments routiers sont maintenant des nœuds, compatibles avec les architectures GNN standard
2. **Gestion mémoire** : L'algorithme en deux passes permet de traiter 22 GB de données avec seulement 8 GB de RAM
3. **Format standardisé** : Les fichiers de sortie sont directement compatibles avec PyTorch Geometric
4. **Labels agrégés** : Les moyennes de temps de trajet sont calculées avec pondération par nombre d'observations

Le script `build_line_graph.py` est prêt à être exécuté sur les données enrichies de la Phase 1.

A Formules Mathématiques

A.1 Nombre d'Arêtes du Line Graph

Pour un graphe orienté $G = (V, E)$, le nombre d'arêtes du line graph est :

$$|E'| = \sum_{v \in V} d_{in}(v) \cdot d_{out}(v) \quad (9)$$

où $d_{in}(v)$ et $d_{out}(v)$ sont les degrés entrant et sortant du nœud v .

A.2 Complexité Algorithmique

Table 6: Complexité des opérations

Opération	Temps	Espace
Construction index	$O(E)$	$O(V + E)$
Génération arêtes	$O(E \cdot \bar{d})$	$O(E')$
Extraction features	$O(E \cdot d)$	$O(E \cdot d)$

où $\bar{d}$ est le degré moyen et d le nombre de features.

B Code Source

Le script complet `build_line_graph.py` est fourni séparément. Les fonctions principales sont :

- `build_adjacency_index()` : Première passe, construction des dictionnaires
- `build_edge_index()` : Deuxième passe, génération du `edge_index`
- `extract_node_features()` : Extraction des features des liens
- `extract_labels_chunked()` : Agrégation des labels par streaming
- `validate_line_graph()` : Vérifications de cohérence